Ch.15 — Motion Control Systems

TTK4190 · Fossen Lecture Notes · Chapter 15

GNC series: Ch.12 Guidance → Ch.8 Vehicle Models → Ch.15 Motion Control → Ch.9 Allocation → Ch.4 Hydrostatics → Ch.2 Kinematics

How the ψ_d and θ_d references that Ch.12's guidance produces actually get turned into fin and thruster commands — and, unusually for this course, Fossen's own worked example for the depth loop is written for a vehicle just like yours.

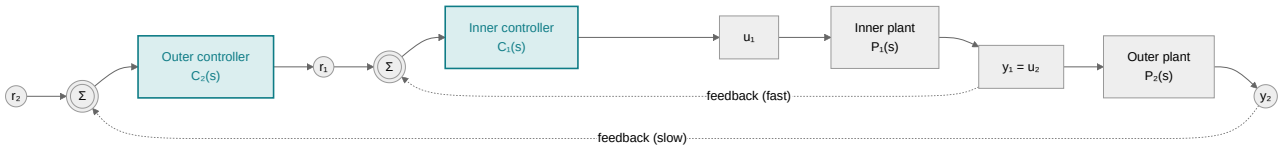
Vehicle: torpedo hull, stern thruster, 4 fins "+", bow tunnel thruster
Applies to: §15.2.2 (heading), §15.2.4 (diving — direct match), §15.2.3 (path-following, adapted)
Skip: §15.1 (ship maneuvering trials), §15.3.6–15.3.7 (DP/mooring), §15.3.3 MIMO Lyapunov (→ Ch.16)

§0 Symbol glossary (new this chapter)

SYMBOL	NAME	WHAT IT PHYSICALLY IS
`K, T`	Nomoto gain, time constant	Two numbers that summarize your yaw response to a fin deflection: r = K/(Ts+1)·δ. Bigger K = more turn rate per degree of fin; bigger T = sluggish, slow to spin up.
`δ_R`	rudder/fin deflection	Reused symbol for whatever produces the yaw moment — for you, the "+" fins commanded for yaw.
`δ_S`	stern-plane deflection	Same fins, commanded for pitch.
`ζ, ω_n`	damping ratio, natural frequency	Standard 2nd-order system knobs. ζ=1 is critically damped (fastest with no overshoot); ω_n sets how fast.
`K_p, K_d, K_i`	PID gains	Fossen derives these directly from ζ, ω_n and actuator limits — not hand-tuned by trial and error.
`W`	bandwidth separation factor	How much faster the inner loop must be than the outer loop in a cascaded design, typically 5–10×. (Unrelated to the weight W from Ch.8 — same letter, different chapter's convention.)
`M_q̇, M_q, M_δS`	hydrodynamic derivatives	Added-inertia, damping and fin-effectiveness coefficients in pitch — the numbers Ch.8/Ch.9's models actually produce.
`I_y`	pitch moment of inertia	Rigid-body inertia about the body y-axis.
`BG_z`	vertical CB–CG offset	Exactly z_g from Ch.8 §7 — the lever arm that makes the hull self-righting. `BG_z·W` is that same restoring stiffness, reappearing here as the "spring" term in the pitch equation.
`K_DC`	DC gain (pitch → depth)	The steady-state ratio between a constant pitch and the resulting depth-rate response; compensated for in the outer depth loop.

§1 Successive loop closure — the general idea

Instead of designing one complicated controller for a coupled multi-state system, close a fast **inner loop** around the part you can measure quickly and directly (e.g. pitch), then wrap a **slower outer loop** around that (e.g. depth) that treats the inner loop as if it were already "solved" ($y_1/r_1 \approx 1$). This is the same nesting pattern used for both case studies below.



Rule of thumb: pick the outer loop's bandwidth $\omega_2 = \omega_1 / W$ with W between 5 and 10, so the inner loop has settled long before the outer loop asks it to move again — that's what makes $y_1/r_1 \approx 1$ a valid approximation.

The gains themselves come from **pole placement**: write the closed-loop characteristic polynomial, decide where you want the poles (via ζ, ω_n), and solve for K_p, K_d directly — no manual tuning loop required as a starting point. Fossen further ties ω_n to actuator saturation: given the maximum control effort u^max and an acceptable step error e_y^max, $\omega_n = \sqrt{(a_0 + b_0 \cdot u^{max}/e_y^{max})}$. This is a genuinely different design flow from "guess gains, run it, tweak" — you specify physical limits and get gains out.

---

§2 Heading autopilot — memorize this pattern

The Nomoto model reduces your yaw dynamics to a single gain/time-constant pair:

```
r = K/(Ts+1)·δ_R          ψ = K/(s(Ts+1))·δ_R + (1/s)d_r
```

r is yaw rate, d_r a disturbance (current, asymmetric drag). PD control law, then pole-placement gains, then the practical PID with the ssa() wrap you already met via the Remus 100 case study in Ch.12:

```
δ_R = −K_pψ·ssa(e_ψ) − K_dψ·ṙ − K_iψ·∫ssa(e_ψ)dτ        e_ψ = ψ − ψ_d

K_pψ = δ_R^max / e_ψ^max        K_dψ = (2ζψωψT − 1)/K        ωψ = √(
(K/T)·(δ_R^max/e_ψ^max) )
K_iψ = (ωψ/10)·K_pψ    (rule of thumb)
```

This is the yaw-axis instantiation of §1's pattern: e_ψ drives the fin deflection directly (no cascaded inner state here — r is fast enough to fold into a single PID). Notice there's no restoring term anywhere in the Nomoto model — yaw has no "spring" pulling it back to a preferred heading. That's exactly the directional-instability point from Ch.12/Ch.15.1: without feedback, a torpedo hull will happily sit at any heading. Pitch, next section, is different.

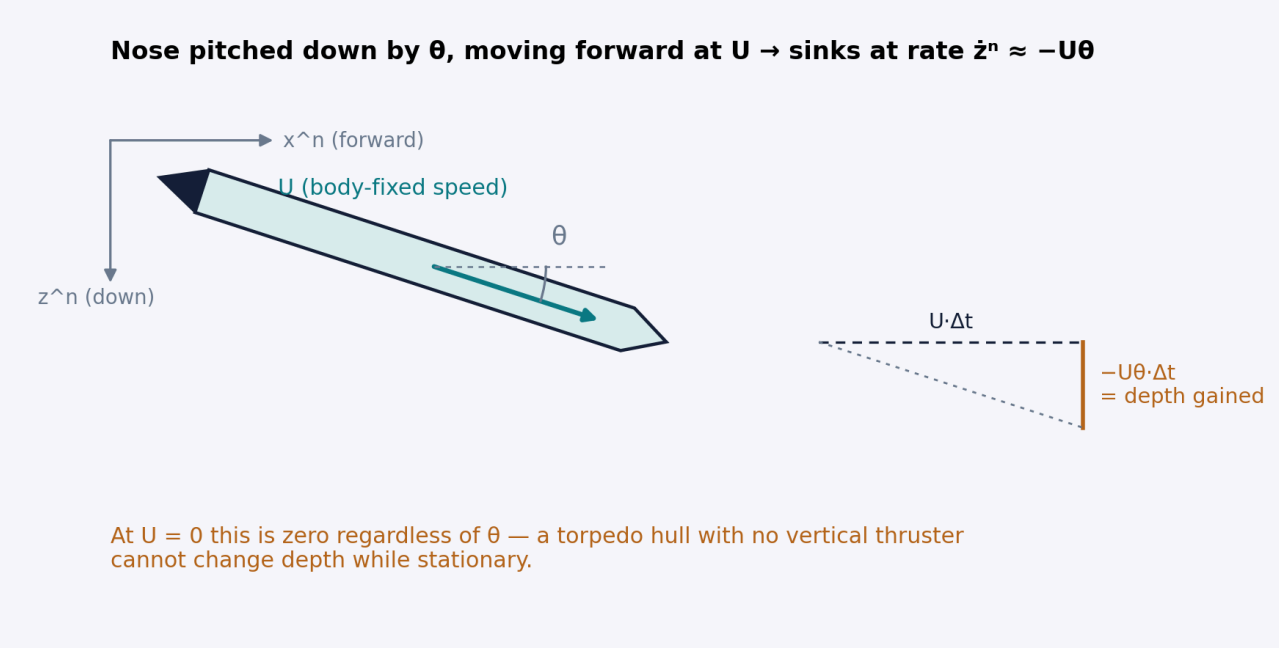
---

§3 Diving autopilot — Fossen's own underwater-vehicle case study

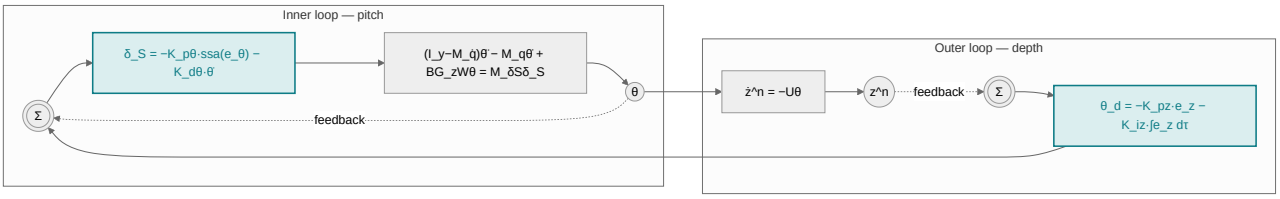
§15.2.4 is written for exactly your topology: pitch controlled by stern dive planes, no separate heave actuator, depth achieved by pitching and moving forward. This is the least adapted section in the whole series — read it close to verbatim.

```
ż^n = −Uθ + d_z

(I_y − M_q̇)θ̈ − M_q·θ̇ + BG_z·W·θ = M_δS·δ_S
```



The `BG_z·W·θ` term is exactly Ch.8 §7's restoring stiffness (z_g·W) showing up again — pitch **is** passively spring-restored (self-righting), unlike yaw. That changes what the pitch autopilot's job actually is: it's shaping the response of an already-stable system, not creating stability from nothing.



Transfer functions and gains (K_DC is the pitch loop's nonunity DC gain, compensated for in the outer PI):

```
θ/δ_S(s) = [M_δS/(I_y−M_q̇)] / [s² + (−M_q/(I_y−M_q̇))s + BG_zW/(I_y−M_q̇)]

K_pθ = δ_S^max/e_θ^max      K_dθ = [2ζθωθ(I_y−M_q̇) + M_q] / M_δS

z/s ≈ −(K_DC·U/s)θ_d       θ_d = −K_pz·e_z − K_iz·∫e_z dτ

K_pz = 2ζzωz/(K_DC·U)       K_iz = ωz²/(K_DC·U)
```

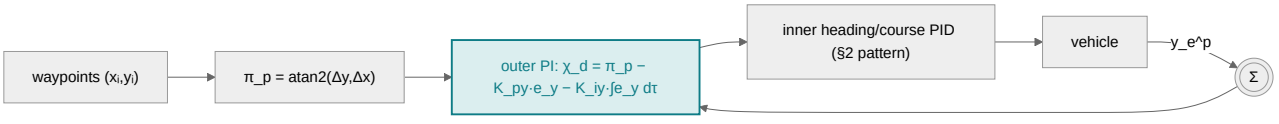
The gains scale with 1/U
K_pz and K_iz both have U in the denominator — the depth loop's effective gain depends on how fast you're moving. This is the control-theory face of the geometric fact above: no forward speed, no depth authority, and gains tuned for cruising speed will be too weak (or unstable if inverted) near zero speed. Worth logging U alongside depth-loop performance during pool tests.

---

§4 Path-following: gluing Ch.12's guidance into this chapter's control

§15.2.3 shows the full loop Ch.12 promised: an outer guidance loop (cross-track error → commanded course) wrapped around the same heading-autopilot inner loop from §2.

```
ẏ_e^p = U·sin(χ_p) ≈ U·χ_p          χ_p = χ − π_p
```



Fossen writes this in terms of course angle χ (needs GPS/COG) — for you, substitute ψ throughout exactly as Ch.12 §5 already established: use the heading-based LOS law, not this course-based one. The structural pattern (outer PI on cross-track error, wrapped around an inner PID heading loop, gains scaled by 1/U in the outer loop exactly as in §3) carries over unchanged. Ch.12's along-track waypoint-switching test is the more robust version of this section's circle-based switching — prefer it.

---

§5 What to skip, and why

TOPIC	WHAT IT ASSUMES	VERDICT
§15.1 ship maneuvering trials (turning circle, zigzag, pull-out, spiral, stopping)	Full-scale ship, rudder sizing and hydrodynamic derivative identification from sea trials.	**Skip** — for a scratch-built AUV you'll get K, T and the pitch derivatives from Ch.8/Ch.9's models or bench/pool system-ID, not sea trials.
§15.3.6 Dynamic Positioning, §15.3.7 Position Mooring	Station-keeping with 3 independent horizontal controls (surge, sway, yaw) and/or a mooring spread.	**Skip** — you have 2 controls and no sway actuator; can't hover regardless.
§15.3.3 MIMO nonlinear PID / Lyapunov backstepping	Full coupled 6-DOF nonlinear control with a Lyapunov-based stability proof.	**Skip for now** — genuinely v2 material; Fossen defers the advanced version of this to Ch.16 (backstepping, sliding mode) himself.
Wind feedforward (§15.3.4, §15.3.6)	A wind sensor and aerodynamic force model.	**Skip** — not applicable underwater; current plays a related role and is handled by Ch.12's ILOS/ALOS instead.

§6 Where this plugs into what you already have

Your existing Pixhawk/ArduSub + custom Python PID is presumably a single-loop heading and depth controller. §2 and §3 don't demand you rebuild that architecture — they give you a principled way to derive its gains from physical parameters (K, T for yaw; I_y, $M_{\dot{q}}$, M_q, $BG_z \cdot W$, $M_{\delta S}$ for pitch) instead of trial-and-error tuning, and a template (§1) for turning it into a cascaded design later if a single loop isn't precise enough. The physical parameters themselves are exactly what Ch.8's spheroid/Myring recipe and Ch.9's allocation chapter produce — this chapter tells you what to do with them once you have them.

§7 Memorize vs. reference

Memorize

- Successive loop closure: fast inner loop, slow outer loop, bandwidth separation $W \approx 5–10$
- Nomoto model $r = K/(Ts+1)\cdot\delta$, and that pole-placement gains come from ζ, ω_n + actuator limits
- ssa() in every PID angle error, still
- Diving autopilot structure: inner pitch loop (already spring-restored via $BG_z \cdot W$) + outer depth loop (gains scale as 1/U)

- $\dot{z}\hat{}n \approx -U\theta$: no forward speed ⇒ no depth authority

Reference only

- Exact Nomoto K,T identification via turning-circle/zigzag/spiral trials
- DP and position-mooring system design
- Wind feedforward equations
- MIMO nonlinear Lyapunov PID (→ Ch.16 when you get there)

Next: Ch.9 — control forces and allocation.